



De La Salle College of Saint Benilde - Antipolo
College of Business
8 L. Sumulong Memorial Circle, Antipolo, 1870
Rizal

Data Structures and Algorithms

BSMM02

Submitted by:

Charles David

S. Parizal

Data Structures and Algorithms

Data Structures and Algorithms (DSA)

Data Structures are ways of organizing and storing data so that it can be accessed and modified efficiently. Different data structures are designed for different purposes. Examples include arrays, linked lists, stacks, queues, trees, and graphs.

Algorithms are step-by-step instructions used to solve a problem or perform a specific task. An algorithm can be used to search, sort, process, or manipulate data stored in a data structure. Data structures and algorithms work together because the way data is organized can affect how efficiently an algorithm can process it (W3Schools, n.d.).

Data Structures and Algorithms (DSA) are important concepts in programming and information technology because computer programs often need to store, process, and retrieve large amounts of information. Choosing an appropriate data structure can make information easier to access, while choosing an appropriate algorithm can reduce the amount of work needed to complete a task.

For an Information Technology or Information Systems student at De La Salle-College of Saint Benilde Antipolo, learning DSA provides a foundation for understanding how software processes information. For example, an information system may need to store student records, product information, transactions, or user accounts. The way these records are organized and searched can affect the performance of the system. DSA therefore helps students understand how to develop programs that process information efficiently.

Linear Data Structure

A Linear Data Structure is a data structure where elements are arranged in a sequential or linear order. The elements can generally be processed one after another from the beginning toward the end. Common examples include arrays, linked lists, stacks, and queues (OpenDSA, n.d.).

The term “linear” refers to the logical arrangement of the elements. However, the way these elements are stored in computer memory can differ depending on the specific data structure. For example, an array allows access through an index, while a linked list connects its elements through links or references.

Linear data structures are useful when data needs to be handled in a specific sequence. For example, a queue can represent students waiting for assistance, while a stack can represent tasks that follow a last-in, first-out order.

For an Information Systems student, understanding linear data structures is useful when designing programs that process records, transactions, requests, or other information in an organized sequence. The appropriate structure depends on how the information needs to be accessed, added, removed, or processed.

One-Dimensional (1D) Array

A One-Dimensional Array is a linear collection of elements that can be accessed using a single index. In many programming languages, array indexing begins at 0. This means that the first element is located at index 0, the second element at index 1, and so on (W3Schools, n.d.).

For example:

Student Scores = [85, 90, 78, 92, 88]

The indexes would be:

Index: 0 | 1 | 2 | 3 | 4

Value: 85 | 90 | 78 | 92 | 88

In this example, **Student Scores[0]** refers to 85, while **Student Scores[3]** refers to 92.

One major advantage of an array is direct access to an element when its index is known. Accessing an array element by index generally takes **$O(1)$ time**, because the location can be calculated directly instead of checking each element one by one.

However, traditional fixed-size arrays require their size to be determined when they are created. If the allocated size is too large, some memory may not be used. If the size is too small, additional storage may be needed. Some programming languages provide dynamic array structures that can grow when additional elements are added.

In an information system, a one-dimensional array can be useful for storing a simple collection of related values, such as student scores, item quantities, or identification numbers.

The Big O Notation

Big O Notation is a mathematical notation used in computer science to describe how the time or space requirements of an algorithm grow as the input size increases. It focuses on the growth rate of an algorithm rather than the exact amount of time it takes on a specific computer (Cormen et al., 2022).

Big O is commonly used to analyze the efficiency of algorithms. It allows developers to compare different approaches and understand how they may perform when the amount of data becomes larger.

It is important to note that Big O describes an asymptotic upper bound. In basic algorithm analysis, it is often used when discussing worst-case growth, but Big O itself is not automatically the same as “worst-case performance.” Best-case, average-case, and worst-case performance can each be analyzed using asymptotic notation.

Common Big O Complexities

$O(1)$ – Constant Time

An algorithm with **$O(1)$** complexity performs a fixed amount of work regardless of how large the input becomes.

For example, accessing an element from an array when its index is known can generally be done in constant time:

Student Scores[3]

The program can directly access the element at index 3 without checking the other elements.

$O(\log n)$ – Logarithmic Time

An algorithm with **$O(\log n)$** complexity reduces the problem size by a factor during each step. A common example is Binary Search, which repeatedly divides the search area into smaller sections.

For example, instead of checking every element in a sorted list, Binary Search can eliminate approximately half of the remaining elements after each comparison.

$O(n)$ – Linear Time

An algorithm with **$O(n)$** complexity performs work that increases approximately in proportion to the number of input elements.

Linear Search is an example. If a list contains 100 elements, the algorithm may need to check up to 100 elements in the worst case. If the list contains 1,000 elements, it may need to check up to 1,000 elements.

$O(n^2)$ – Quadratic Time

An algorithm with **$O(n^2)$** complexity has a growth rate related to the square of the input size.

A basic implementation of Bubble Sort can have $O(n^2)$ worst-case time complexity because it may repeatedly compare elements using nested iterations. As the amount of data increases, the number of operations can increase significantly.

Big O is useful when developing information systems because systems may eventually handle thousands or millions of records. An algorithm that works well with a small dataset may become inefficient when the amount of data becomes much larger.

For example, searching through a small list of student records using Linear Search may be acceptable. However, when a system contains a much larger number of records, a more efficient searching method may reduce the amount of work required. Understanding Big O helps Information Systems students evaluate the efficiency of different algorithms before implementing them.

Basic Searching Algorithms

Searching algorithms are used to locate a specific value, record, or item within a collection of data.

Two basic searching algorithms are **Linear Search** and **Binary Search**.

Linear Search

Linear Search, also called Sequential Search, is a searching algorithm that checks elements one at a time, starting from the beginning of a collection. It continues until the target value is found or all elements have been checked (OpenDSA, n.d.).

How It Works

Consider the following list:

[15, 8, 23, 42, 10]

If the target value is **42**, Linear Search checks the elements in this order:

15 → 8 → 23 → 42

The algorithm stops when it finds 42.

If the target is not in the list, the algorithm checks all elements before determining that the value is not present.

Requirement

One advantage of Linear Search is that the data does **not need to be sorted**. This makes it useful for small or unsorted collections.

Time Complexity

- **Best Case: $O(1)$** – The target is the first element.
- **Average Case: $O(n)$** – The target is somewhere in the collection.

- **Worst Case: $O(n)$** – The target is the last element or is not present.

A simple information system could use Linear Search to find a particular student ID in a small, unsorted list of student records.

For a small dataset, checking each record one by one may be practical. However, as the number of records increases, the number of comparisons can also increase. This shows why understanding algorithm efficiency is important when designing information systems.

Binary Search

Binary Search is a searching algorithm that repeatedly divides a **sorted** collection into smaller sections to locate a target value. Instead of checking every element, it examines the middle element and eliminates the half of the collection where the target cannot exist (OpenDSA, n.d.).

Requirement

The collection must be **sorted** before Binary Search can be correctly applied.

For example:

[10, 20, 30, 40, 50, 60, 70]

If the target is **60**, Binary Search first checks the middle value, which is 40.

Since 60 is greater than 40, the algorithm eliminates the values smaller than 40 and continues searching the remaining section:

[50, 60, 70]

The process continues until the target is found or there are no remaining elements to check.

Time Complexity

- **Best Case: $O(1)$** – The target is the first middle element checked.
- **Average Case: $O(\log n)$** – The search repeatedly reduces the search area.
- **Worst Case: $O(\log n)$** – The search continues dividing the remaining elements until the target is found or determined to be absent.

Binary Search can be useful when an information system repeatedly searches through a collection of records that is already sorted by a searchable value. For example, a system may maintain records in sorted order by an identification number and use a searching method that takes advantage of this ordering.

However, Binary Search is not automatically appropriate for every information system. The data must meet its requirements, especially being sorted and stored in a structure that supports efficient access to the middle positions. If the data is not sorted, sorting may need to be performed first.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). The MIT Press.

OpenDSA. (n.d.). *Binary search*. Virginia Tech.

<https://opensa-server.cs.vt.edu/ODSA/Books/Everything/html/BinarySearch.html>

W3Schools. (n.d.). *Data structures and algorithms*. <https://www.w3schools.com/dsa/>

W3Schools. (n.d.). *DSA arrays*. https://www.w3schools.com/dsa/dsa_data_arrays.php